Solving Dynamic Programming Problems using Code Generation

Michael Lin, Matthew Samuels, Charles Anton Sibal

March 11, 2026

Worcester Polytechnic Institute

A Major Qualifying Project Report
Submitted in partial fulfillment of the requirements for the
Degree of Bachelor of Science

Project Advisor: Professor George Heineman

This report represents work of WPI undergraduate students submitted to the faculty as evidence of a degree requirement. WPI routinely publishes these reports on its web site without editorial or peer review. For more information about the projects program at WPI, see https://www.wpi.edu/Academics/Projects

Abstract

Dynamic Programming (DP) is an algorithmic and mathematical paradigm used to solve problems by optimizing overlapping sub-problems. However, translating the structure of a DP problem into a runnable implementation is a tedious and error-prone process that distracts from the problem's structure. This project aims to streamline this process by developing a framework capable of generating complete Java solutions to DP problems using a high-level metamodel. Our metamodel, in conjunction with a parsing engine, can be used to represent a DP problem and turn that representation into runnable Java code. The code can be generated with both bottom-up and top-down strategies without any additional work. The code generation was done using a Scala framework called CoGen; this comes with the benefit that additional languages can be added in the future without altering the structure of our work. In addition to the model, we compiled a repository of hand-coded DP problem solutions in Java which are organized by problem family and implementation strategy with similar problems grouped together.

Contents

Abstract 2

1 Introduction 6

2 Background 10
- 2.1 Dynamic Programming 10
  - 2.1.1 Top-Down Dynamic Programming 10
  - 2.1.2 Bottom-Up Dynamic Programming 11
- 2.2 Domain Specific Languages 11
- 2.3 DP Problems and Solutions 12
- 2.4 Code Generators 13

3 Methodology 15
- 3.1 Project and Environment Structure 15
  - 3.1.1 Handcoded DP Solutions 15
  - 3.1.2 Different DP solutions 15
  - 3.1.3 DSL Solutions 15
- 3.2 Developing a Model 17
- 3.3 Developing an Engine 17

4 Design 18
- 4.1 Hand-coding Solutions to DP Problems 18
- 4.2 Categorizing DP Problems 19
- 4.3 Dynprog Representations of DP Solutions 21
- 4.4 Authoring Conceptual Representations of DP Problems through CoGen 21
- 4.5 Generating Fibonacci Code 24
  - 4.5.1 File Structure 26
  - 4.5.2 Initial Generated Solutions 27
- 4.6 Factoring out Helper Methods 27
- 4.7 Generalizing Solutions by Family 28
  - 4.7.1 Top-Down Problems 28
  - 4.7.2 Linear Problems 29
  - 4.7.3 Problems with Two Sequences 30
  - 4.7.4 Grid Problems 30
  - 4.7.5 Graph 30
  - 4.7.6 Knapsack 31
- 4.8 Modeling DP Problems 31
  - 4.8.1 Linear Problem Model 31
  - 4.8.2 Model Workflow 37
  - 4.8.3 Model limitations 38

4.9 **EnhancedModel** . . . 38
4.9.1 Enhanced Model Workflow . . . 41
4.9.2 Comparison to Dynprog . . . 42
4.9.3 Limitations of Enhanced Model . . . 42

5 Conclusion . . . **44**
5.1 **Testing Code** . . . 44
5.2 **Code Size Comparison** . . . 45
5.3 **Revisiting Families** . . . 45
5.4 **Future Work** . . . 45
5.4.1 Research into Different Problem Structures . . . 46
5.4.2 Additional Definition capabilities . . . 46
5.4.3 Additional Expression capabilities . . . 47
5.4.4 Additional Array capabilities . . . 47
5.4.5 Solution Generation in Other Languages . . . 47
5.4.6 Path Traversal of Dynamic Programming Problems . . . 48
5.4.7 Researching Solution Path Retrievals . . . 48

References . . . **51**

List of Figures

Figure 1 Recursion tree for $Fib(5)$ 10
Figure 2 A one-dimensional DP array used to compute $Fib(4)$ via tabulation. The value at index 4 is calculated in $O(1)$ time by directly summing the previously stored values from indices 2 and 3. 11
Figure 3 Generated Fibonacci Scala solution 22
Figure 4 Relationships between different paradigms 24
Figure 5 Architecture for CoGen, with Fibonacci Sample Problem 24
Figure 6 A visualization of Cantor's pairing function.[7] 29
Figure 7 cloc results 45
Figure 8 A traceback matrix for the LongestCommonSubsequence algorithm. Image sourced from [33]. 48

1 Introduction

Solutions to many computational problems often utilize naive or brute-force approaches with large time or space complexity. *Dynamic Programming* (DP) is an algorithmic paradigm designed to overcome this problem by identifying similar and repeating subproblems to avoid extraneous calculations.[3]

Consider the LONGESTCOMMONSUBSEQUENCE (LCS) problem where you are given two strings and are asked to compute the length of the longest common subsequence. A subsequence of a string is a sequence of characters drawn from the string in order from left to right. A subsequence is different from a sublist because a sublist must be contiguous whereas a subsequence may contain letters that are not adjacent to each other in the original string. For example, given string s = `ACTGAT`, the string t = `ATA` is a subsequence of s because each of the characters in t appears in left-to-right order within s. For LCS, there are two strings, s_1 and s_2, and the goal is to find the length of the longest common subsequence, lcs, where lcs is a subsequence of both s_1 and s_2.

```java
public class AllSubsequences {

    String source;
    Stream.Builder<String> builder = Stream.builder();

    public AllSubsequences(String s) {
        this.source = s;
    }

    public Stream<String> generate() {
        generateAll(source, "");
        return builder.build();
    }

    private void generateAll(String rest, String current) {
        // Base case: Empty 'rest' string means current is a subsequence
        if (rest.equals("")) {
            builder.add(current);
            return;
        }

        // Recursive step: Try with and without first character
        generateAll(rest.substring(1), current + rest.charAt(0));
        generateAll(rest.substring(1), current);
    }

    // Usage: java AllSubsequences S1 S2
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println ("Usage: java AllSubsequences S1 S2");
            System.exit(0);
        }
        if (args[1].length() < args[0].length()) {
            String tmp = args[0]; args[0] = args[1]; args[1] = tmp;
        }

        Stream<String> stream1 = new AllSubsequences(args[0]).generate();
        String result = new FindLongest(args[1]).process(stream1);
        System.out.println ("LCS is " + result);
    }
}
```

Listing 1: Naive approach to LONGESTCOMMONSUBSEQUENCE

For simplicity, let's say that s_1 and s_2 both contain n characters. The naive approach in Listing 1 computes all possible subsequences in s_1 and checks whether each subsequence is contained in s_2, recording the longest one found. The problem is that for a string of

length n, there are 2^n possible subsequences. In the worst case, you would have to check each of these subsequences for inclusion in s_2. Listing 2 contains the $O(N)$ implementation that determines whether a subsequence *sub* exists within a *target* string. This solution is an $O(N2^N)$ algorithm for computing the length of the longest subsequence.

```
import java.util.*;
import java.util.stream.Stream;

public class FindLongest {
    String target, longest;
    public FindLongest(String t) { target = t; }

    public void findmax(String sub) {
        int i = 0;  // index into sub
        int j = 0;  // index into target

        while (i < sub.length() && j < target.length()) {
            if (sub.charAt(i) == target.charAt(j)) {
                i++;   // move on to next position in sub
            }
            j++;       // Always move to next position in target
        }

        if (i == sub.length() && i > longest.length()) { longest = sub; }
    }

    public String process(Stream<String> subsequences) {
        longest = "";
        subsequences.forEach(this::findmax);
        return longest;
    }
}
```

Listing 2: Naive approach to LONGESTCOMMONSUBSEQUENCE

This naive implementation wastes tremendous time in recomputing partial solutions over and over again. Dynamic programming stores the results of smaller subproblems to avoid such wasted recomputation. Listing 10 contains a sample Dynamic Programming solution to LCS. As is typical for Dynamic Programming problems, the resulting answer is an integer: additional work is required to recover a sequence of the computed length. Note that there may be multiple such sequences, and any one of the given length would be a suitable solution.

The DP implementation technique is encoded in a nested `for` loop which composes together solutions from smaller problems until the final result is computed. These solutions must be computed in a specific order for the logic in lines 21–29 to be correct.

The `retrieve` method at line 5 in Listing 4 recovers the solution by traversing the entries in `dp`. This code computes the solution *in reverse* by looking for the final decision in the solution, which recorded whether the rth character in string $s1$ equals the cth character in $s2$, which makes it part of the longest common subsequence. If not, the solution is found by considering the solution to the subproblem of the LCS between $s1[0..r-1]$ and $s2[0..c]$ or the subproblem for the LCS between $s1[0..r]$ and $s2[0..c-1]$.

```
package org.dp.bottomUp.twoSequences;

public class LongestCommonSubsequence {
    String s1, s2;
    int len1, len2;
    int[][] dp;

    public int solution(String s1, String s2) {
        /** Initialization */
        this.s1 = s1;
```

```java
        this.s2 = s2;
        len1 = s1.length();
        len2 = s2.length();
        dp = new int[len1 + 1][len2 + 1];

        /** Base Cases: dp[0][...] and dp[...][0] are 0 by default */
        for(int r = 1; r <= len1; r++) { dp[r][0] = 0; }
        for(int c = 1; c <= len2; c++) { dp[0][c] = 0; }

        /** Iterative solution */
        for (int r = 1; r <= len1; r++) {
            for (int c = 1; c <= len2; c++) {
                if (s1.charAt(r - 1) == s2.charAt(c - 1)) {
                    dp[r][c] = dp[r - 1][c - 1] + 1;
                } else {
                    dp[r][c] = Math.max(dp[r - 1][c], dp[r][c - 1]);
                }
            }
        }

        /** Return bottom right element */
        return dp[len1][len2];
    }
}
```

Listing 3: Hand-coded LONGESTCOMMONSUBSEQUENCE

```java
package org.dp.bottomUp.twoSequences;

public class LongestCommonSubsequence {

    // .... added method
    public String retrieve() {
        int lcsLength = dp[len1][len2];
        char[] path = new char[lcsLength];

        int r = len1, c = len2;
        int index = lcsLength - 1;

        while (r > 0 && c > 0) {
            if (s1.charAt(r - 1) == s2.charAt(c - 1)) {
                path[index] = s1.charAt(r - 1);
                index--;
                r--;
                c--;
            } else if (dp[r - 1][c] > dp[r][c - 1]) {
                r--;
            } else {
                c--;
            }
        }

        return new String(path);
    }
}
```

Listing 4: Hand-coded LONGESTCOMMONSUBSEQUENCE

This solution is an iterative *bottom-up* solution to LCS. This problem can also be solved in a recursive *top-down* solution.

```java
package org.dp.topDown.twoSequences;

import java.util.*;

public class LongestCommonSubsequence {
    String s1, s2;
    int len1, len2;

    public LongestCommonSubsequence(String s1, String s2) {
        this.s1 = s1;
        this.s2 = s2;
        this.len1 = s1.length();
```

```
        this.len2 = s2.length();
    }

    public int solution() {
        return helper(len1, len2)
    }

    public int helper(int r, int c) {
        if(r == 0 || c == 0) {
            return 0;
        } else if(s1.charAt(r - 1) == s2.charAt(c - 1)) {
            return helper(r - 1, c - 1) + 1;
        } else {
            return Math.max(helper(r, c - 1), helper(r - 1, c));
        }
    }
}
```

Listing 5: Hand-coded, naive, and top-down LONGESTCOMMONSUBSEQUENCE

Despite the power of DP as an algorithmic technique, translating a problem's structure into a correct and runnable implementation is a non-trivial and error-prone task. Indeed, often during a technical code interview, applicants are presented with a dynamic programming scenario and asked to complete the solution from scratch.

Current approaches to generating DP solutions require manual derivation of the recurrence relationship and sometimes-tedious implementation and testing for each new problem. This low-level and repetitive effort distracts from the more abstract goal of capturing the problem's structure.

This research aims to develop a framework for solving Dynamic Programming problem from a code-generation metamodel such that one can capture the abstract structure of a problem and generate its solutions in various languages, such as Java or Scala. Since understanding the COGEN metamodel is essential to this approach, documenting its design and our usage of it is a key sub-goal of the project.

2 Background

This section outlines the theoretical foundation of our work. We begin by examining the principle we aimed to investigate—Dynamic Programming, an algorithmic technique that generates efficient solutions by breaking down complex problems into smaller subproblems. We then discuss Domain-Specific Languages (DSLs), which are specialized for specific problem domains and enable concise and expressive program representations. Finally, we explore common aspects of various DP problems and their solutions, highlighting the potential of utilizing DSLs to represent and solve DP problems in a more abstract and systematic manner.

2.1 Dynamic Programming

DP is an algorithm optimization technique that solves complex problems by breaking them down into simpler, overlapping subproblems. Results of already solved subproblems are reused to avoid redundant computations. There are two main approaches to DP: top-down, in which an algorithm solves the problem recursively and stores the results of the subproblems; and bottom-up, where subproblems are solved iteratively and build up to the final solution.

DP problems appear in several fields such as operations research, economics, bioinformatics, and artificial intelligence. DP algorithms are also famously used in the interview process for computer science jobs to test an applicant's problem solving skills. By creating an optimized method for generating solutions DP problems, we can optimize unnecessary time spent in these fields.

2.1.1 Top-Down Dynamic Programming

Top-down Dynamic Programming involves recursively breaking down a problem and calculating solutions to sub-problems as needed. While the theory is straightforward, an inherent challenge commonly arises: the solution to a single sub-problem may be required multiple times. This redundancy is negligible for smaller inputs, but the potential for exponential time complexity and redundant computations grows rapidly with problem size. See the search space for FIBONACCI [37]:

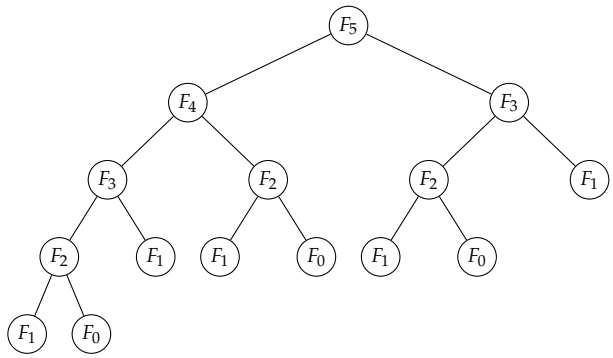

Figure 1: Recursion tree for $Fib(5)$

Including base cases, $Fib(5)$ requires fifteen individual function calls. As the input scales, this redundancy becomes severe; to calculate $Fib(20)$, the sub-problem $Fib(3)$ alone is evaluated over 2500 times.

To resolve this inefficiency, top-down approaches utilize an optimization technique called *memoization*. Memoization caches the results of function calls in a data structure and returns the cached result when the same inputs occur again. In FIBONACCI, for example, once $Fib(2)$ is computed for the first time, its value is stored. Subsequent calls for $Fib(2)$ bypass the recursive steps, retrieving the stored value in $O(1)$ time, thus reducing the overall time complexity from $O(2^n)$ to $O(n)$.

2.1.2 Bottom-Up Dynamic Programming

Inversely, bottom-up approaches to DP start with the default cases, repeatedly solving **all** subsequent cases in a specific order. Solutions to larger sub-problems are found by utilizing the results of smaller sub-problems. Typically, a DP matrix is utilized to maintain and record values.

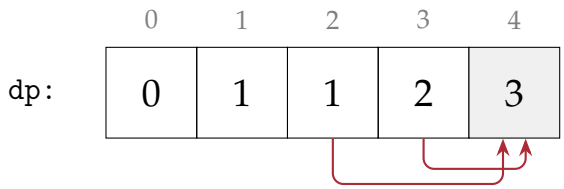

Figure 2: A one-dimensional DP array used to compute $Fib(4)$ via tabulation. The value at index 4 is calculated in $O(1)$ time by directly summing the previously stored values from indices 2 and 3.

While the application of tabulation to the FIBONACCI sequence is relatively straightforward, this technique serves as a foundational and highly versatile tool for optimizing performance across a broad range of more complex Dynamic Programming algorithms.

2.2 Domain Specific Languages

A Domain-Specific Language (DSL) is a type of programming language designed for a specific task or domain, rather than for general-purpose use like Python. Due to their specialized focus, DSLs are typically highly expressive, allowing users to describe solutions using vocabulary specific to the domain. This often results in concise and easily readable programs. Common examples of DSLs include HTML, SQL, and regular expressions (Regex).

However, DSLs tend to also have drawbacks. Due to their specific use-cases, DSLs tend to lack flexibility and potential for usage outside their intended domains. Smaller DSLs also tend to be wrappers of other languages. Therefore, one would need to learn the syntax, keywords, and built-in functions of the parent language to be able to fully leverage the DSL's power.

Few DSLs exist for the purpose of modeling DP problems; Dynprog is one such example [18]. It allows users to specify a problem as a recursive computation, then

using the recursive relationship to fill out and return a table. Working knowledge of R is required to utilize Dynprog, as it is built using R functions.

2.3 DP Problems and Solutions

To better understand Dynamic Programming and its applications, we researched and analyzed many different examples. As expected, they all posses the key quality DP problems have: optimal problem structure containing overlapping sub-problems. The way the overlapping sub-problems appear, however, differ from example to example. We noticed that there were five major categories.

- **Linear/One Sequence problems** involve building up a sequence of values that are consolidated to get the final solution. Each value in the sequence is dependent on multiple previous values; DP is used to save values in the sequence to be reused in computations.
  - FIBONACCI [37]
  - DECODEWAYS [10]
  - BRAINPOWER [31]
  - JUMPGAME2 [14]
  - LONGESTINCREASINGSUBSEQUENCE [8]
  - LONGESTVALIDPARENTHESES [25]
  - MAXSUBARRAY [11]
  - MINCOSTCLIMBINGSTAIR [20]
  - MINCOSTTICKET [22]
  - PERFECTSQUARES [29]
  - TRIBONACCI [26]
- **Grid Problems** involve making decisions at each cell based on computations made previously for adjacent cells. Typically, the base cases can be found in the lowest coordinate (0, 0) and valid state transitions happen in non-decreasing order.
  - COUNTSQUARES [6]
  - MAXIMALSQUARE [19]
  - MINIMUMPATHSUM [23]
  - TRIANGLE [32]
  - UNIQUEPATHS [35]
- **Problems with Two Sequences** involve computing an optimal alignment or transformation cost between two ordered data structures. Each subproblem represents the relationship between prefixes of the two sequences.
  - DISTINCTSUBSEQUENCES [2]

- LongestCommonSubsequence [2]
- MinimumDeleteSum [21]
- MinimumEditDistance [36]
- MinTimeColorfulRope [24]
- NeedlemanWunschSequenceAlignment [27]
- ShortestCommonSupersequence [17]
- UncrossedLines [34]

- **Graph** calculations can be optimized using Dynamic Programming. They can be seen as extensions of the solution methodologies derived from grid problems, where state transitions only appear between adjacent nodes. The key change is that nodes that have already been visited must be recorded to avoid infinite loops.

- **Knapsack** problems involve a limited amount of space and require solving sub-problems with less space. DP is used to avoid recomputing solutions to subproblems, which are then combined for the final answer.
  - Knapsack [4]
  - DoubleKnapsack [4]
  - OnesAndZeroes [28]
  - TwoKeysKeyboard [1]

Besides the naive solution, which calculates each sub-problem every time, there are two main ways to reach a solution to a DP problem: bottom-up, and top-down. Bottom-up solutions start at a problem's base case and iterate up to the solution, tabulating solutions to **every** sub-problem. Top-down solutions, on the other hand, recursively break down the main problem until a base case is reached.

2.4 Code Generators

There are abstract syntax trees (AST) defined for all programming languages. While these are commonly used for compiling or interpreting a language, they can also be used to assemble a computer program using the AST programming interface. One of the most popular and widely used ASTs for Java is `javaparser` [13].

```
CompilationUnit cu = new CompilationUnit();
cu.setPackageDeclaration("org.dp.oneSequence");

ClassOrInterfaceDeclaration fibClass =
  cu.addClass("Fibonacci", PUBLIC);
fibClass.addField("int", "n");

fibClass.addConstructor(PUBLIC)
 .addParameter("int", "n")
 .setBody(new BlockStmt()
  .addStatement(new ExpressionStmt(new AssignExpr(
   new FieldAccessExpr(new ThisExpr(), "n"),
   new NameExpr("n"), ASSIGN)));

IntegerLiteralExpr[] lits = { new IntegerLiteralExpr("0"),
```

```
  new IntegerLiteralExpr("1"), new IntegerLiteralExpr("2") };

fibClass.addMethod("helper", PUBLIC)
  .addParameter(new Parameter(PrimitiveType.intType(), "i"))
  .setType("int")
  .setBody(new BlockStmt()
   .addStatement(new IfStmt(new BinaryExpr(new NameExpr("i"), lits[0], EQUALS),
    new BlockStmt().addStatement(new ReturnStmt(lits[0])), null))
   .addStatement(new IfStmt(new BinaryExpr(new NameExpr("i"), lits[1], EQUALS),
    new BlockStmt().addStatement(new ReturnStmt(lits[1])), null))
   .addStatement(new ReturnStmt(new BinaryExpr(
     new MethodCallExpr("helper").addArgument(new BinaryExpr(new NameExpr("i"),
      new IntegerLiteralExpr("1"), MINUS)),
     new MethodCallExpr("helper").addArgument(new BinaryExpr(new NameExpr("i"),
      new IntegerLiteralExpr("2"), MINUS)),
     PLUS))));
 fibClass.addMethod("compute", PUBLIC)
  .setType("int")
  .setBody(new BlockStmt()
   .addStatement(new ReturnStmt(
    new MethodCallExpr("helper").addArgument(new NameExpr("n")))));
```

Listing 6: Using `javaparser` API to assemble FIBONACCI implementation

The code fragment in listing 6 produces the implementation of FIBONACCI as shown in Listing 7.

Listing 7: Fibonacci Example

```
package org.dp.oneSequence;
public class Fibonacci {
    int n;
    public Fibonacci(int n) {
        this.n = n;
    }
    public int helper_topdown(int i) {
        if (i == 0) { return 0; }
        if (i == 1) { return 1; }
        return helper_topdown(i - 1) + helper_topdown(i - 2);
    }

    public int compute() {
        return helper_topdown(n);
    }
}
```

As you can see, the 36 lines of (rather dense) `javaparser` logic produces the 16 lines of java code that implements FIBONACCI. Programming the AST through its Java interface typically leads to Java code as you see here, which contains a dense arrangement of chained invocations. For this project, we never considered using `javaparser` to generate the DP solutions, but instead chose to use the COGEN code generation framework written in Scala [5] as described in Section 4.4.

3 Methodology

3.1 Project and Environment Structure

The pipeline that we used to implement DP solutions with meta-programming started concrete and progressively got more abstract. We started by hand coding solutions to the example problems that we each found.

The three DP problems solved were PASCAL, LONGESTCOMMONSUBSEQUENCE, and JUMPGAME.

This gave us both a way of understanding the problems themselves as well as a goal for our code generation to create functionally equivalent code. The next step was to create the meta program and generate that code. While this was a level of abstraction above the actual java solution, it still involved specifying a solution to a single problem that would need to be factored out in later steps. The next progression was to look at multiple meta solutions and find what they have in common. When the repeated behaviors were located, we factored them out into helper functions to streamline the code generation process. The final step was to create a direct translation from the problem to the solution. This took the form of something similar to a domain specific language, like dynprog, turning a recursive definition of the problem and implementing it fully in Java.

3.1.1 Handcoded DP Solutions

In order to develop a COGEN framework to generate full solutions for DP problems, we first had to attain a working knowledge of Dynamic Programming as a paradigm. To accomplish this, we researched and found several DP problems, compiling their solutions and test cases into a GitHub repository. The main goals of this process were to divide DP problems into general families and identify what methodologies the solutions had in common to factor them out later during metamodel development.

3.1.2 Different DP solutions

During our investigation of different types of DP solutions, we noticed that the differences between them arise primarily from implementation details rather than fundamental structural variations. Although DP problems can appear diverse, ranging from grid-based path finding to optimization and sequence alignment, their core concept of breaking a problem into smaller subproblems remains the same. This observation suggests that the process of solving DP problems can be systematically abstracted and standardized as long as their underlying structure remains consistent across problem types.

3.1.3 DSL Solutions

Our ultimate goal of streamlining the process of generating solutions to DP problems is similar to the goal of certain Domain Specific Languages (DSLs). By studying DSLs and understanding how to use them to generate solutions, we can help visualize what our final product might look like. Utilizing Dynprog, a superset DSL of R, we made Dynprog solutions for each of our DP examples. In doing so, we found some of the strengths and limitations of Dynprog and all DSLs. Dynprog works by defining each value in

the solution matrix in terms of simpler solutions and defining a range of values for the solution to be on. For simple DP problems, it's a very efficient approach that minimizes code and simplifies the solving process. For more complex solutions, Dynprog became cumbersome to use and required us to bend the solution to fit with the Dynprog format. Dynprog struggles with problems that required an inner loop to define each value as that required using higher order functions within Dynprog for the solution to be calculated.

The following is a simple DP solution for Pascal numbers made using dynprog:

```
library(dynprog)

pasc <- {
    P[n,1] <- 1
    P[1,m] <- 0
    P[n,m] <- P[n-1,m-1] + P[n-1,m]
} %where% {
    n <- 1:20
    m <- 1:20
}
```

Listing 8: Dynprog `Pascal.R`

This implementation is straightforward, and the translation from concept to specific definition is clear. One of the goals with our product is to ensure that simple examples like this can be easily translated. It's with simple DP problems like this that Dynprog is most effective.

Other problems are not so trivial, for example LONGESTINCREASINGSUBSEQUENCE:

```
LISnums <- c(10, 9, 2, 5, 3, 7, 101, 18)

LIS <- max({
    S[1] <-1
    S[n] <- 1 + max(c(0, S[which(LISnums<LISnums[n])]))
} %where% {
    n <- 1:length(LISnums)

})
```

Listing 9: Dynprog `LIS.R`

The way this solution works is through a complicated nesting of higher order functions. The base case for n==1 is simple. The goal of the recursive case is to find the previous subproblem solution with the best value but only look at indices where the number at that location is less than the current number, that way the subsequence is always increasing. The way this is done is through `which(LISnums<LISnums[n])`. This higher order function loops over each element of LISnums and finds the indices of the ones where the value is smaller than the current value. This array is then used as an array access from `S` which gets the eligible subproblem solutions. The max call finds the best possible result, or 0 if there are none. 1 is added to increase the length by taking the current number. The entire result is passed into a max to find the largest possible subsequence length.

This solution is a degree of separation away from the problem. It is not immediately obvious how this code represents the solution to Longest Increasing Subsequence. Understanding how it works requires an understanding of higher order functions and the R languages specific syntax for them. It is likely impossible to sufficiently express DP solutions without higher order functions or loops, so incorporating them into our final

model is essential. Another challenge highlighted by this solution is the translation from the input to the generated solution. Dynprog has solutions formulated in functional language, where specific values are defined in terms of functions. This lends itself to the top down approach of Dynamic Programming but would be harder to convert to bottom up solutions which require loops. Overall Dynprog has been excellent at illumination the potential challenges of encoding DP solutions.

3.2 Developing a Model

The creation of a model for representing DP problems is the ultimate culmination of abstracting details and finding similarities between problems. A model provides a structure that should be strong enough to represent any DP problem with the minimum amount of information. The model is also standardized so it can be passed into a parser to generate runnable code.

This creates a division between the two goals of the project, a model that can represent any DP problem, and a parsing engine that takes in a model and an implementation strategy and generates the solution code. For the creation of the model, we looked at our hand-coded solutions and found what was consistent in all of them. The parts that were the same would be automatically generated by the model parser. The parts that were different were used in the model as a way of specifying which problem we were representing. Once the first version of the model was created we imagined trying to recreate the code from just the information in our model representation. If there wasn't enough there, it meant we needed to add something new to represent that part of the problem we were missing. Additionally, if there was a problem we couldn't represent, we weren't being general enough, and certain information we assumed was universal was actually problem specific. In these cases we needed to add more fields to our model to create generality.

There were also times in our model development when we realized that there was repetition in the representations. For our model to be efficient we didn't want to have any redundant information. If one field taking on a certain value meant another field had to take on a value, our model was bloated and we had to remove or simplify fields.

3.3 Developing an Engine

The creation of the model was abstract and had very loose implementation. Each model representation is mostly a chaining of nested expressions to represent the problem, without many rules as to what it needed to look like. The parsing engine is where the rules came in to play. Unlike, the model, the engine required extensive use of CoGen to actually generate the desired code.

The two main components of the engine were the part that parses the expressions and the part that puts everything together to generate the code. Parsing expressions involves recursively evaluating them and converting them from model code to CoGen generator code. The rest of the parser acts as a framework to combine expressions and read each field from the model. This code depended on the implementation method and would either use for loops or recursive calls for bottom up and top down respectively.

4 Design

This project aims to generate full Java solutions to existing DP problems using a code generator called CoGen. The approach to completing this project can be broken down into several tasks:

- Manually code in Java several DP problems across different DP domains.
- Identify the structures that are common across multiple solutions and what varies.
- Develop initial CoGen solutions for individual DP problems.
- Refactor CoGen code to identify helper methods that reduce the size of the code base.

4.1 Hand-coding Solutions to DP Problems

After collecting and assigning our initial set of solutions to DP problems to families, we decided to develop our own solutions in Java using the ones we collected as reference. Within each Java solution file named after the intended problem to model, we wrote `solution` methods containing the solution, identifying sections that contain code that initialize variables, maximize the solution structure, and return the solution itself.

```java
package org.dp.bottomUp.twoSequences;

public class LongestCommonSubsequence {
    String s1, s2;
    int len1, len2;
    int[][] dp;

    public int solution(String s1, String s2) {
        /** Initialization */
        this.s1 = s1;
        this.s2 = s2;
        len1 = s1.length();
        len2 = s2.length();
        dp = new int[len1 + 1][len2 + 1];

        /** Base Cases: dp[0][...] and dp[...][0] are 0 by default */
        for(int r = 1; r <= len1; r++) { dp[r][0] = 0; }
        for(int c = 1; c <= len2; c++) { dp[0][c] = 0; }

        /** Iterative solution */
        for (int r = 1; r <= len1; r++) {
            for (int c = 1; c <= len2; c++) {
                if (s1.charAt(r - 1) == s2.charAt(c - 1)) {
                    dp[r][c] = dp[r - 1][c - 1] + 1;
                } else {
                    dp[r][c] = Math.max(dp[r - 1][c], dp[r][c - 1]);
                }
            }
        }

        /** Return bottom right element */
        return dp[len1][len2];
    }
}
```

Listing 10: Hand-coded LongestCommonSubsequence without Path Retrievability

An issue arose when attempting to retrieve the path of decisions from the solution–since all the variables were created locally in the solution methods, they were inaccessible to any retrieve method we wished to create. In order to widen the scope of visibility for the necessary information, we made them attributes of the class and instantiated them in the `solution` method. Listing 11 shows the `retrieve` method that retrieves the computed solution as a String.

```java
package org.dp.bottomUp.twoSequences;

public class LongestCommonSubsequence {

    // .... added method
    public String retrieve() {
        int lcsLength = dp[len1][len2];
        char[] path = new char[lcsLength];

        int r = len1, c = len2;
        int index = lcsLength - 1;

        while (r > 0 && c > 0) {
            if (s1.charAt(r - 1) == s2.charAt(c - 1)) {
                path[index] = s1.charAt(r - 1);
                index--;
                r--;
                c--;
            } else if (dp[r - 1][c] > dp[r][c - 1]) {
                r--;
            } else {
                c--;
            }
        }

        return new String(path);
    }
}
```

Listing 11: Hand-coded solution path reconstruction method for LONGESTCOMMONSUBSEQUENCE2

4.2 Categorizing DP Problems

After creating solutions for each problem, we grouped the problems and abstracted their key attributes to better understand their underlying structure. Initially, we separated the problems into top-down and bottom-up approaches, and then further categorized them into groups such as single sequence, grid, and knapsack. However, as we progressed, we realized that this method would require storing each problem in two separate folders—one for top-down and one for bottom-up—which could lead to confusion. Therefore, instead of creating two files for each problem, we decided to include both the top-down and bottom-up solutions in a single file.

```java
package org.dp.oneSequence;

import org.dp.IBottomUp;
import org.dp.ITopDown;
/**
 * Name: Fibonacci Number
 * Description:
 * Given a positive integer n, find the nth Fibonacci number.
 *
 * The Fibonacci series is a sequence where a term is the sum of previous
 * two terms. The first two terms of the Fibonacci sequence are 0 followed by 1.
 * The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21.
 */
```

```java
public class Fibonacci implements ITopDown, IBottomUp {
    int n;
    int[] dp;
    public Fibonacci(int n) {
        this.n = n;
        dp = new int[n + 1];
    }
    public int helper_topdown(int i){
        if(i == 0){
            return 0;
        }
        if (i == 1){
            return 1;
        }
        return helper_topdown(i - 1) + helper_topdown(i - 2);
    }

    public int helper_bottomup() {
        if (n == 0) return 0;
        if (n == 1) return 1;

        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

    public int solution_bottomup(){
        return helper_bottomup();
    }

    public int solution_topdown(){
        return helper_topdown(n);
    }

    public int solution(){
        return solution_topdown();
    }
}
```

Listing 12: Example reprosotory code for FIBONACCI

The code above provides an example of the FIBONACCI problem. At the top of the file, the problem name and description are defined. The problem input is encapsulated within the class, so each instance requires creating a new object. There are two implementations provided: one using the top-down approach and the other using the bottom-up approach. The final `solution()` method serves as an interface to facilitate easier testing and access to the implementations.

- problem with both topdown and bottomup implementation
  - **One Sequence**: CoinChange, Decodeways, JumpGame, LongestIncreasing-Subsequence, LongestValidParentheses, MatrixChainMultiplication, MaxSub-array, MinCostClimbingStair, Palindrome
  - **Integer**: Fibonacci, CountNumberUniqueDigits, MaximumProductCutting, PerfectSquares, Tribonacci
  - **Integers**: EggDrop
  - **Strings**: Interleaving Strings, ThreeStrings

  - **Two Sequences**: LongestCommonSubsequence, MinimumDeleteSum, MinimumEditDistance, MinTimeColorfulRope, ShortestCommonSuperSequence, WildCarePatternMatching, Wordbreak
  - **Grid**: Bellnumber, DiceThrow, MaixmumSquare, Triangle, UniquePath
  - **Knapsack**: Knapsack
  - **Graph**: MaximalIndependentSetPath
- problem with only topdown implementation
  - **One Sequence**: DeleteAndEarn, HouseRobber
  - **Integer**: TwoKeysKeyboard
  - **Grid**: Pascal
  - **Knapsack**: DoubleKnapsack, MultipleKnapsack, OneAndZeros
- problem with only bottomUp implementation
  - **One Sequence**: BrainPower, MinCostTicket
  - **Two Sequences**: DistinctSubsequences, SequenceAlignment, UncrossedLine
  - **Grid**: CountSquares, MinPathSum
  - **Graph**: Dijkstra

4.3 Dynprog Representations of DP Solutions

Though it is incomplete and difficult to use at times, Dynprog does provide a starting point for representing Dynamic Programming problems. Some of the key ideas we took away from Dynprog were the use of boundaries for which the problems were defined and the list of cases for how to evaluate a subproblem. For the boundaries, we modified this by turning it into a set of values derived from the arguments passed into the function instead of being hard-coded into the model. It makes sense that this needed to be changed as Dynprog is used to solve one instance of a DP problem while our model is used to generate code that can solve any input to a DP problem. We also modified the cases. While Dynprog uses pattern matching to figure out which case to use, our model would be transformed into Java code so we used if-else statements.

4.4 Authoring Conceptual Representations of DP Problems through CoGen

CoGen is a code generation framework written in Scala [5]. In this section, we demonstrate how to write a CoGen specification for computing the nth Fibonacci number, from which we can then generate code in different programming languages and paradigms, Java and Scala in particular. Table 1 contains the generic set of fundamental syntax concepts that CoGen requires when working with Abstract Syntax Trees (AST).

CoGen generates code by programmatically manipulating ASTs using a generic API whose abstract capabilities are realized by language-specific implementations (currently Java and Scala are supported). Any code to be generated belongs in a specific *context*.

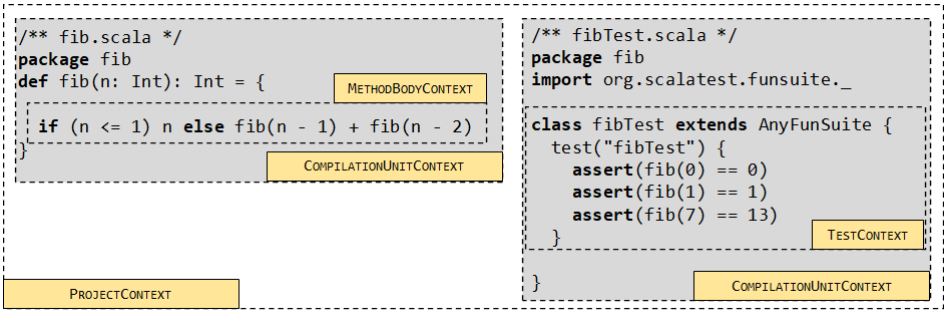

Figure 3: Generated Fibonacci Scala solution

The following four contexts are universal across all programming languages. Figure 3 highlights code fragments to demonstrate examples of code generated for these contexts:

- CompilationUnitContext represents a unit of code that resides in a single source code file
- ProjectContext groups together any number of CompilationUnitContext and represents a directory of source code files
- MethodBodyContext contains a sequence of code statements
- TestContext contains a sequence of test cases

For each of these contexts there is a corresponding collection of *capabilities*. For example, textttMethodBodyCapabilities, provides a number of abilities, including:

- Set the return type for a method body
- Get the arguments for a method body
- Add an import (to the enclosing `CompilationUnitContext`) if needed for code in a method body
- Construct an expression by applying a method to a sequence of arguments

`CompilationUnit`	A single file of source code
`Import`	An import declaration to pull in library code
`Expression`	An expression
`Type`	A type annotation or declaration
`Statement`	A single statement of source code
`Name`	A valid name
`UnitTest`	A single test case

Table 1: Common syntax concepts

- Reify an instance of a primitive type

Each generic capability is modeled as a `Command`, following the Command design pattern [12]. Listing 13 contains two language-independent command definitions in CoGen, one to set the return type for a `MethodBodyContext` to a specific `Type`, and another to retrieve the arguments for a `MethodBodyContext`. This listing also contains the fragment of the generator that manipulates AST fragments for the Scala language.

```
// language-independent command definitions
case class SetReturnType[Type](tpe: Type) extends Command {
  type Result = Unit
}

case class GetArguments[Type, Name, Expression]() extends Command {
  type Result = Seq[(Name, Type, Expression)]
}

// How to process SetReturnType for Scala language
implicit val canSetReturnTypeInMethodBody :
  Understands[MethodBodyCtxt, SetReturnType[Type]] =
    new Understands[MethodBodyCtxt, SetReturnType[Type]] {
      def perform(context: MethodBodyCtxt, cmd: SetReturnType[Type]) :
        (MethodBodyCtxt, Unit) = {
        val updatedMethod = context.method match {
          case definition: Defn.Def =>
            definition.copy(decltpe = Some(cmd.tpe))
          case decl: Decl.Def =>
            decl.copy(decltpe = cmd.tpe)
        }

        (context.copy(method = updatedMethod), ())
      }
    }
```

Listing 13: Command definitions with Scala generator to set return type for a method definition or declaration

Every `Command` represents information delivered to a code generator for processing. Each `Command` defines a `Result` type, which can depend upon the kind of command and is customized as needed. The interpretation of a `Command` can change based upon the context in which the command was created. The actual contexts themselves depend on the code generation. In CoGen there are over 50 specialized `Command` case classes that represent the full scope of these generic capabilities.

Within the `MethodBodyCtxt` in Listing 13, this code shows that it understands how to interpret a `SetReturnType` commmand by creating a copy of the current `MethodBodyCtxt` and altering the definition of the `decltpe` (the *declaring type*, an attribute in the AST node) to be `cmd.tpe`, the type associated with the `Command`.

Figure 4 identifies four high-level specifications for the AST manipulations needed for different programming paradigms. Each of these contexts comes with defined capabilities appropriate for the context:

- `Functional` adds `TypeContext` according to functional programming languages
- `ObjectOriented` adds the `ClassContext` and `ConstructorContext` to represent that constructors are special methods
- `ParametericPolymorphism` adds the `TypeParameter` context, allowing new capabilities to the `MethodBodyContext`

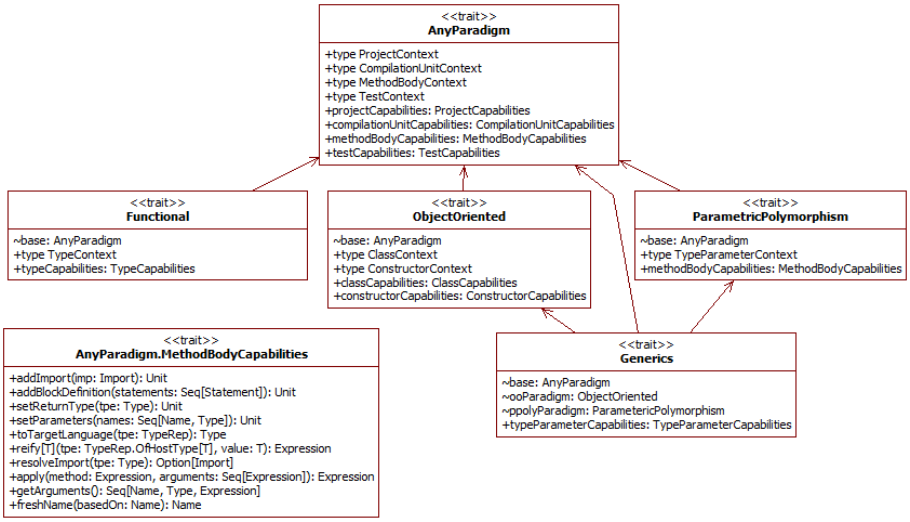

Figure 4: Relationships between different paradigms

- `Generics` applies the capabilities of parameteric polymorphism to `ObjectOriented`

These high-level specifications for a code generator contain the specific capabilities represented as commands to be interpreted within the actual language-specific code generators. To ensure proper syntax, these language-specific code generators rely on customized AST manipulations for the appropriate language to construct and manipulate AST tree fragments, a significant improvement over simply using string manipulation to assemble generated code. CoGen uses JavaParser [13] to generate Java code fragments and Scalameta [30] to generate Scala code fragments. Future programming languages can be incorporated by implementing the appropriate Language Implementation Provider (LIP).

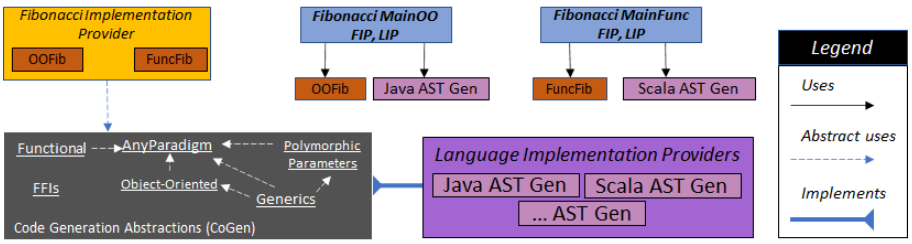

Figure 5: Architecture for CoGen, with Fibonacci Sample Problem

4.5 Generating Fibonacci Code

As shown in Figure 5, two Fibonacci Implementation Providers (FIP) are designed, an `OOFib` to generate code following the object-oriented paradigm while `FuncFib` generates

code in the functional paradigm. Two Language Implementation Providers (LIP) are available, `Java AST Gen` and `Scala AST Gen`. To generate the full code, a main function selects the desired FIP and LIP and requests the code to be generated, as we now show.

The logic to compute the nth Fibonacci number is straightforward, based on the timeless formula $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. Initial conditions are that $F_0 = 0$ and $F_1 = 1$. The code we wish to generate is the functional approach (in Scala) shown in Figure 3, which contains both the actual `fib(n:Int)` function as well as some test cases to ensure correctness.

```
1  def make_fibonacci(): Generator[MethodBodyContext, Expression] = {
2    for {
3      /** Method will have single n:Int parameter and return Int. */
4      intType <- toTargetLanguageType(TypeRep.Int)
5      _ <- setParameters(Seq(("n", intType)))
6      _ <- setReturnType(intType)
7
8      /** Get arguments to function and constant values. */
9      one <- reify(TypeRep.Int, 1)
10     two <- reify(TypeRep.Int, 2)
11     args <- getArguments()
12
13     /** Logic for whether n ≤ 1. */
14     (name,tpe,n) = args.head
15     le1 <- ffi.le(n, one)
16
17     /** Form two subexpressions (n-1) and (n-2). */
18     n_1 <- ffi.sub(n, one)
19     n_2 <- ffi.sub(n, two)
20
21     /** Apply twice, to fib(n-1) and fib(n-2); add together. */
22     func <- findMethod(Seq(fibName))
23     fn_1 <- apply(func, Seq(n_1))
24     fn_2 <- apply(func, Seq(n_2))
25     addExpr <- ffi.add(fn_1, fn_2)
26
27     res <- ifThenElse(le1,
28                 Command.lift(n),          /** If n ≤ 1 then n */
29                 Seq.empty,                /** Empty "else if" */
30                 Command.lift(addExpr))    /** Else fib(n-1)+fib(n-2) */
31   } yield res
32 }
33
34 def make_unit(): Generator[CompilationUnitContext, Unit] = {
35   for {
36     _ <- addMethod("fib", make_fibonacci())
37   } yield ()
38 }
39
40 def make_project(): Generator[ProjectContext, Unit] = {
41   for {
42     _ <- addCompilationUnit(make_unit(), "fib")
43     _ <- addCompilationUnit(addTestSuite("fibTest", make_test()), "fibTest")
44   } yield ()
45 }
```

Listing 14: Language logic for Fibonacci in functional paradigm

Lines 4-6 ensure that the `MethodBodyContext` will have a single parameter, `n`, of integer type and the return type for the method will also be an integer. The `toTargetLanguage` command is especially worthy of attention, since it bridges the gap between this language-independent code specification and the final generator that produces (in this case) Scala code. The concept of an integer, `TypeRep.Int`, will be converted into the actual `Type` in Scala, `Int`, to represent integers. Lines 9-11 reify two constants that will be useful and retrieves the arguments to the function. Since there is only one argument,

line 14 retrieves the code expression associated with the first (and only argument) `n`. Line 15 constructs the expression `(n` $\leq$ `1)` which will be used as the guard to an `if` statement in line 27. Lines 18-19 represent the expressions `n-1` and `n-2`. Lines 22-24 retrieve the function representing the invocation of `fib` and creates two sub-expressions, one for `fib(n-1)` and the other for `fib(n-2)`. Line 25 constructs an expression that sums these two expressions together, as expected for the Fibonacci series. Lines 27-30 create the final result, an `if` statement that returns `n` if `n` $\leq$ `1`, otherwise it returns the computed addition of the two sub-expressions.

`make_fibonacci()` in Listing 14 is a function with type `Generator[MethodBodyContext, Expression]` which means this function returns a generator that can produce an `Expression` within a `MethodBodyContext`. The **for ... yield** structure collects together a sequence of commands, binding the last result of a command, using it in the next command. Internally this is implemented by a sequence of `flatMap` operations, where each `flatMap` operation takes the state of the context prior to the command, and then the command as a function returning the next state. Instead of directly executing the commands, we abstractly collect `flatMaps` as a single nested data type which will later be iterated over to interpret it with respect to the meaning given by the Language Implementation Provider (LIP).

CoGen includes a Foreign Function Interface (FFI) that defines common concepts that exist across different programming languages. Listing 14, for example, includes `le` (less-than or equal), `add` to add two expressions, and `sub` to subtract one expression from another. The full interface supports additional arithmetic operations (such as modulo or multiplication), working with arrays, assertions, boolean values, printing to a console, lists, trees, floating point arithmetic, time operations, exceptions and strings. The FFI interface can be extended as needed to include additional capabilities required for additional domains.

The `make_project()` builds up a collection of transactions containing `Commands` constructed within the code. `make_unit()` produces a `CompilationUnitContext` with a sequence of commands to create a method named `fib` with the method body constructed in `make_fibonacci()`. `make_project()` adds the `CompilationUnit` to the project. Once the transaction of commands is executed, the result will be a collection of ASTs which can then be formatted using available "pretty print" functionality provided by the ASTs; the code can then be written directly to disk.

This brief introduction to CoGen only covers the essential concepts, showing how to use the API to specify, in a language-independent manner, the desired code logic to generate.

4.5.1 File Structure

In CoGen, there are a number of boilerplate files necessary to invoke the code generation commands as shown earlier. A `MainJave` file is the entry point to the generation and calls other methods to generate the code. The Provider file has important methods to be used by the other files that help specify what code to generate. Each of these files uses imports from other project files that supply the behavior for CoGen. This Fibonacci example was manually coded as a sample case to validate CoGen. The project engaged in the

process of working with CoGen even while it was undergoing constant development.

4.5.2 Initial Generated Solutions

We began our work in CoGen by choosing one DP problem each and writing code to generate that solution using CoGen commands. This process acclimated us to CoGen and gave us a surface level understanding of how it worked. These solutions also gave us a jumping off points for our later work of optimization and helped us think about what could be made simpler in the process of writing higher level code.

4.6 Factoring out Helper Methods

Solutions to DP problems typically follow a consistent structure—initialization, state transition (optimization), and return—and problems across different families often exhibit similar patterns in how these stages are implemented. In order to reduce code redundancy, we decided to factor similar lines of code out into helper methods.

The first step in this process was identifying what each helper function needed to produce—in other words, determining the function signatures we intended to implement. Our main goal was for the helper functions to generate coherent and purposeful statements, or sequences of statements, so those functions' signatures took the form

```
Generator[MethodBodyContext, Statement]
Generator[MethodBodyContext, Seq[Statement]].
```

However, there were some cases where we only wanted to simplify the creation of value expressions. Therefore, the functions' signatures were

```
Generator[MethodBodyContext, Expression].
```

We started with simple helper methods such as `max` and `plus equals`. These both gave us experience in creating helper methods and could be used in the future to make compound helper methods with more complex behavior. For example, `make_for_loop()` uses `plus_equals()` to increment the iterator. In this way, the problem of making helper methods is divided into more manageable pieces that can be implemented individually.

```
def make_for_loop(iterator: Expression,
                  guard: Expression,
                  update: Expression, body: Seq[Statement]):
  Generator[paradigm.MethodBodyContext, Statement] = {
    import paradigm.methodBodyCapabilities._
    import impParadigm.imperativeCapabilities._
    for {
      while_loop <- whileLoop(
        guard,
        for {
          _ <- addBlockDefinitions(body)
          updated_it <- assignVar(iterator, update)
          _ <- addBlockDefinitions(Seq(updated_itr))
        } yield ()
      )
    } yield while_loop
}
```

Listing 15: CoGen code to generate loop

The ultimate goal of our work with helper methods is to create a method that processes a *model* representing how each solution is built from simpler solutions and returns a complete generator for that solution. For example, to define the solution to Fibonacci, one should only need to specify that F(0)=0, F(1)=1, and F(n)=F(n-1)+F(n-2), as well as the general form that the problem takes, in this instance it is a single sequence. In this way, the specification is at its most fundamental form, and our final program should be able to fill in the repetitive parts of the implementation. The most challenging part of implementing DP solutions is specifying that relation.

Each of the categories of DP problems have different forms and different helper methods work better for each of them. The following section outlines the form that each category takes, which helper methods can contribute to their simplification, and the general process of fully abstracting them to their core essence.

4.7 Generalizing Solutions by Family

After gaining simple modularity through the creation of helper functions, we sought to abstract the structure of a DP family set.

4.7.1 Top-Down Problems

Top-down problems all share a general format that can be abstracted out and made to generate automatically. One of the main components of a top-down solution is *memoization*. Memoization is a DP technique that stores the result of function calls with the same inputs to increase performance. There are several ways to implement this: for our work, we used a `HashMap` that maps from a key derived from the inputs to the function to a value for the return from the function. The code below shows an example of memoization in computing Pascal's triangle.

```java
public class Pascal {
  java.util.HashMap<Integer,Integer> memo = new HashMap<>();

  public int solution(int r, int c) {
    /** Base Cases */
    if (c==0) { return 1; }
    if (r==0) { return 0; }

    /** Cantoring pairing */
    int s = r + c;
    Integer key = (s*(s+1))/2 + r;

    if (!memo.containsKey(key)) {
      memo.put(key, solution(r-1, c-1) + solution(r-1, c));
    }

    return memo.get(key);
  }
}
```

Listing 16: Memoization Implementation

.

An issue that arises with memoization is that of encoding the key for the hash table. Pascal takes two integer arguments, but we need a single value to act as the key for the table. The naive approach would be to put the two values into an array and use that as the key, however java arrays do not override the equals method so this will

not work for retrieving the value. The solution to this issue is in something called a pairing function. This is a function that takes two integer values and combines them into a unique integer key. Attempting to simply add or multiply the arguments together would fail as a pairing function because certain keys would be repeated. The function used in the Pascal example is the Cantor pairing function. It works by noticing a couple of things.

- The diagonal that a pair of values is on can be found by adding them together.
- Each diagonal has one greater length than the previous one.
- The sum of the first n natural numbers is $\frac{n(n+1)}{2}$

With these details in mind, the Cantor pairing equation can be understood.

$$\frac{(a+b)(a+b+1)}{2} + a$$

The first term represents the number of elements in all of the previous diagonals, the second term represents which element of the current diagonal.

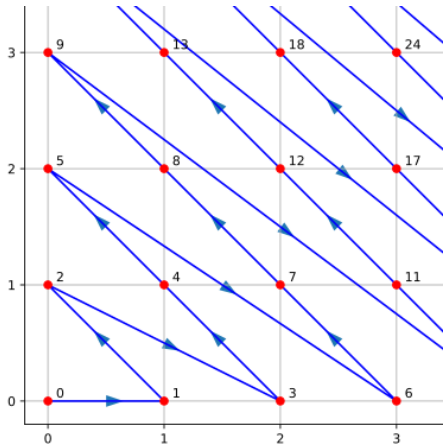

Figure 6: A visualization of Cantor's pairing function.[7]

The result is a unique key defined by the two inputs. A helpful property of pairing functions is that they can be chained together to form a unique key for an arbitrary number of elements so long as the number of elements is consistent.

4.7.2 Linear Problems

Bottom Up Linear problems usually involve a single loop calculating answers one at a time until the solution is found. There are several components that can be factored out such as the initialization of the DP array, the loop, a potential maximization function within the loop, and returning the final element from the DP array. what is left is the function that calculates new answers from previous answers as well as smaller initialization details such as how the length of the DP array is found.

4.7.3 Problems with Two Sequences

Two Sequence problems in DP involve optimizing a feature over two inputs S1 and S2, utilizing a two-dimensional state DP[I][J] to represent the solution for prefixes of increasing length of both sequences. Typically, solution values of base cases are initialized in the 0th row and column, as they correspond to scenarios where one or both of the sequences are empty.

Once the base cases are established, the remaining cells are filled by following the recurrence relation defined by the problem's constraints. The value of the state DP[I][J] in question is derived by making a decision based on the characters of the sequences S1[I], S2[J], and the values of the previously computed neighboring states:

- DP[I - 1][J]: representing the solution when the first sequence is shortened
- DP[I][J - 1]: representing the solution when the second sequence is shortened
- DP[I - 1][J - 1]: representing the solution when both sequences are shortened simultaneously–typically used when processing both characters at the same position together, whether they match or not

The final answer to the original problem is typically found in the bottom-right entry of the matrix, DP[N][M], where N and M are the lengths of the two sequences.

4.7.4 Grid Problems

In Grid DP problems, the input is typically a 2D grid where the value at each index GRID[I][J] represents some value–such as a cost, reward, or obstacle–that influences the relationship between subproblems. Unlike Two Sequence problems that involve two separate inputs, the grid serves as a single input.

Although the structure of grid problems allows us to store subproblem values directly in the input grid, we chose to maintain a separate DP table for clarity and consistency with problems of other families. The value of DP at each state corresponds to the solution of a subgrid spanning from the top-left corner to its respective bottom-right corner.

- DP[I - 1][J]: represents the solution of a subgrid of size I - 1 by J
- DP[I][J - 1]: represents the solution of a subgrid of size I by J - 1
- DP[I - 1][J - 1]: represents the solution of a subgrid of size I - 1 by J - 1

4.7.5 Graph

In Graph DP problems, the input is typically a graph where nodes represent states and edges represent transitions between them. Similar to Grid problems, solutions of Graph subproblems depend on the solutions of adjacent nodes. The input is a graph where nodes represent states and edges represent transitions between them. Unlike grid problems where each cell has at most three predecessors, the number of adjacent states to consider varies per node depending on the structure of the graph.

Another key distinction is the potential presence of cycles. In grid problems, the top-to-bottom, left-to-right filling order naturally ensures dependencies are resolved before they are needed. In graphs with cycles, this ordering is not immediately obvious, and techniques such as memoization are often used to handle circular dependencies.

4.7.6 Knapsack

Knapsack problems seem complex but are actually not very different from other DP problems. Knapsack problems can be identified by having list of items to take or not take and a limited resource that must be iterated over. The most important part of that definition is that of the limited resource adding an extra dimension to the problem. Problems such as coin change might initially seem like knapsack problems, but yield in relatively simple ways that can't be done to knapsack problems.

The most common knapsack problem is 0/1 knapsack. This problem represents a choosing to take or leave a set of items while staying under a given weight and maximizing a value. As an NP-hard problem, knapsack cannot be solved quickly. The given weight of the problem is a factor in its runtime because it becomes a dimension of the DP array or a variable to recurse over. However, once the weight of the problem is understood to represent an extra dimension, all knapsack problems can be solved in a similar manor to two-sequence problems. Where one sequence is the list of items, and the other sequence is the list of weights less than or equal to the given weight.

4.8 Modeling DP Problems

Because of the highly-stylized nature of DP problems, we envisioned it would be possible to design a meta-modeling capability to design the model for a DP problem. From the model, it should be possible to design a program that processes the model to generate solutions in Java to numerous DP problems. After manually implementing about a dozen DP problems, we had sufficient experience to start modeling the DP domain. The model must be accurate and specifically reference individual DP problems without being harder to develop than the actual DP solutions themselves.

Our models use nested expressions, assembled together to form the final DP representation. The construction of the models were similar to the construction of abstract syntax trees in programming language development.

4.8.1 Linear Problem Model

The first model we created represented linear DP problems, as they are the simplest.

```
class Model(
  val problem:String,
  val bounds: List[Expression],
  val cases: List[(Option[Expression], Expression)]
)
```

Listing 17: Linear Model

.

This model shown in Listing 17 takes 3 inputs to specify the problem:

- PROBLEM: A scala string for the name of the problem, this will be used when generating the class name for the final file.
- BOUNDS: An expression for how the program will calculate the bounds over which to iterate. It can be as simple as passing in the input of the function, or something more complex like taking the length of a string. Nested expressions allow arbitrary complexity.
- CASES: A list of expression pairs for the different cases of the problem. The first expression in each pair is either a boolean expression for whether the case will be used for a given problem, or None for a default case. The second expression in each pair is the solution to the problem in that case.

The starting point is the `Expression` abstract concept, implemented as a Scala trait. There are many different kinds of expressions:

- `LiteralInt(val literal: Int)` – represents a single integer
- `LiteralString(val literal: String)` – represents a single String
- `LiteralChar(val literal: Char)` – represents a single character
- `AdditionExpression(val left: Expression, val right: Expression)` – represents the addition of two expressions
- `SubtractionExpression(val left: Expression, val right: Expression)` – represents the subtraction of two expressions
- `MultiplicationExpression(val left: Expression, val right: Expression)` – represents the multiplication of two expressions
- `DivisionExpression(val left: Expression, val right: Expression)` – represents the division of two expressions
- `EqualExpression(val left: Expression, val right: Expression, val tpe:ArgumentType = org.combinators.models.IntegerType())` – represents an equality check for two expressions
- `MaxExpression(val left: Expression, val right: Expression)` – selects the maximum value from a pair of expressions
- `MinExpression(val left: Expression, val right: Expression)` – selects the minimum value from a pair of expressions
- `AndExpression(val left: Expression, val right: Expression)` – represents comparing two children expressions using the logical **and** operator
- `OrExpression(val left: Expression, val right: Expression)` – represents comparing two children expressions using the logical **or** operator

- `TernaryExpression(val condition:Expression with BooleanExpression, val trueBranch:Expression, val falseBranch:Expression)` – represents a conditional expression that evaluates to `trueBranch` if `condition` holds, and `falseBranch` otherwise

There are several `Expression` specializations that are directly related to the subproblem or the implementation:

- `ArgExpression(val whichArg: Int, val name: String, val argType: ArgumentType, val itArgName: String)` – selects the specified argument from the list of arguments for the DP problem
- `SubproblemExpression(val args: Seq[Expression])` – represents the solution of a subproblem in the same domain, based on a different set of arguments, so it must be a sequence to cover nearly every case
- `IteratorExpression(val iteratorNumber:Int, val variable: String)` – frequently DP problems are applied to integer domains, where there are different bounds representing a range. When there are multiple dimensions to the problem (as might be found in LONGESTCOMMONSUBSEQUENCE for example) there would be two mutually independent expressions, one to range over the characters in string s_1 while the other iterates over string s_2.

For a comprehensive list of all supported expressions, please refer to the IL-DP branch of the COGEN repository [5].

Computing Fibonacci numbers can be represented as following:

$$\begin{cases} F_0 = 0 \\ F_1 = 1 \\ F_n = F_{n-1} + F_{n-2} \text{ for } n > 1 \end{cases} \tag{1}$$

Listing 18 represents the model for the above concepts. The `bound` represents that there is only a single argument to this function, which will be defined as n for the `IteratorExpression`.

```
val zero = new LiteralInt(0)
val one = new LiteralInt(1)
val two = new LiteralInt(2)

val bound = List(new ArgExpression(0))

val n = new IteratorExpression(0)
val nm1 = new SubtractionExpression(n, one)
val nm2 = new SubtractionExpression(n, two)

val Fib = new Model(
  "Fibonacci",
  bound,
  cases = List(
    ( Some(new EqualExpression(n, zero)), zero ),
    ( Some(new EqualExpression(n, one)), one ),
    ( None, new AdditionExpression(
        new SubproblemExpression(Seq(nm1)),
```

```
            new SubproblemExpression(Seq(nm2)))
        )
    )
)
```

Listing 18: Linear Model

.

The central element of this model is `IteratorExpression` which represents the one (and only) iterator for the problem. This represents n, the argument to `fib(n)`. From this `IteratorExpression` we are able to derive the two `SubProblemExpression` objects that represent `fib(n-1)` and `fib(n-2)`.

The ordering of the cases is important. Each case is defined by a tuple (`Option`, `Expression`). There are three cases to consider, namely, when n is 0, 1 or some other number. In reality, this model could be more effective if the `EqualExpression` were replaced with an `LessThanOrEqualExpression`, so it could properly handle when $n < 0$.

There should only be one case that has `None` as its first entry, and this should be the last one in the list. The other cases (as many as are required for the problem) appear first, and represent the *Base Case(s)* for the problem. In this case, $F_0 = 0$ is represented by `(Some(new EqualExpression(n, zero)), zero)`. This part of the model represents the logic that says "when `n` is zero, the result of `fib(n)` is zero." Similarly there is a second case to represent that "when `n` is one, the result of `fib(n)` is one."

Observe how the `LiteralInt` expressions are used to capture constants essential to the definition of the domain. This model captures the essence of the Fibonacci problem.

This model can be passed into a method that generates to code for a top-down implementation in Java. Listing 19 contains a reasonable implementation in Java. In fact, this is the generated code.

```
public Integer helper(Integer n) {
  if (Integer.valueOf(n).equals(0)) {
    return 0;
  } else if (Integer.valueOf(n).equals(1)) {
    return 1;
  } else {
    return (this.helper((n - 1)) + this.helper((n - 2)));
  }
}
```

Listing 19: Generated Code for Fibonacci

For top-down, recursive DP implementations, the performance is often inefficient because subproblems are recomputed numerous times. The common technique to improve performance is to use *memoization* to record the result of each computation, to avoid recomputing subproblems. Listing 20 contains the code that uses a `HashMap<Integer,Integer> memo` which stores the results of each computation to prevent duplicate executions.

```
public Integer helper(Integer n) {
  Integer result;
  if (this.memo.containsKey(n)) {
    return this.memo.get(n);
  }
  if (Integer.valueOf(n).equals(0)) {
    result = 0;
  } else if (Integer.valueOf(n).equals(1)) {
    result = 1;
  } else {
```

```
    result = (this.helper((n - 1)) + this.helper((n - 2)));
  }

  this.memo.put(n, result);
  return result;
}
```

Listing 20: Generated Code for Fibonacci Optimized with *memoization*

The Scala implementation in CoGen that processes the model for top-down implementation is shown in 21.

```
def process_inner_helper(result_var: Option[Expression], model:Model):
  GeneratorMethodBodyContext, Option[Expression]] = {
    val real_cases = model.cases.filter(p => p._1.isDefined)
    val first_case = real_cases.head
    val tail_cases = real_cases.tail
    val elseCase = model.cases.filter(p => p._1.isEmpty)

    for {
      _ <- make_compute_method_signature()
      intType <- toTargetLanguageType(TypeRep.Int)
      _ <- setReturnType(intType)
      inner <- explore(first_case._1.get)

      all_rest <- forEach(tail_cases) { next_case =>
        for {
          next_cond <- explore(next_case._1.get)
          next_exp <- explore(next_case._2)
        } yield (next_cond, expand(result_var, next_exp))
      }

      ifstmt <- impParadigm.imperativeCapabilities.ifThenElse(
        // condition of first if
        inner
        ,
        // statements for that first if
        for {
          resexp <- explore(first_case._2)
          av <- if (memo) {
            impParadigm.imperativeCapabilities.assignVar(result_var.get, resexp)
          } else {
            impParadigm.imperativeCapabilities.returnStmt(resexp)
          }
          _ <- addBlockDefinitions(Seq(av))
        } yield None
        ,
        // collection of (condition, block) for all of the remaining cases
        all_rest
        ,
        // terminating 'else' takes the elseCase and adds it last
        Some(for {
          result_exp <- explore(elseCase.head._2)
          av <- if (memo) {
            impParadigm.imperativeCapabilities.assignVar(result_var.get, result_exp)
          } else {
            impParadigm.imperativeCapabilities.returnStmt(result_exp)
          }
          _ <- addBlockDefinitions(Seq(av))
        } yield ())
      )

      _ <- addBlockDefinitions(Seq(ifstmt))
    } yield None
  }
```

Listing 21: Top Down interpreter for model

The purpose of `process_inner_helper` is to translate the `Model` into working Java code. This is accomplished by generating an `if .. then .. else` block with an

ordered sequence of conditions, terminating in a final `else`. The CoGen logic (lines 9–11) ensures the signature of the `helper(Integer n)` method has a single integer parameter.

Lines 14–18 converts each of the remaining specified cases into an "`else if (condition) { body }`". The body of each of these `if` and `else` statements must decide whether to (a) return a value, as would be done without memoization; or (b) to set `result` to the computation so it can later be stored in the `memo HashMap`.

The code depends on two special methods: `expand()` and `explore()`. `explore` takes in an `Expression` from the model and converts it into a `Generator[MethodBodyContext, Expression]`, which means it can be converted by CoGen into operational code. The method is recursive and each subclass of `model.Expression` has a `case` to be evaluated. For example, `AdditionExpression` is converted into the mathematical addition of two expression, as interpreted by the `left` and `right` components of the `AdditionExpression`. Each `SubproblemExpression` results in the execution of the `helper` method on the designated arguments for that subproblem. `IteratorExpression` acts on one of the parameters to the method and retrieves the name of that variable as required. As additional `Expression` subtypes are added, this method will change to generate the corresponding code.

```
1  def explore(expr : org.combinators.model.Expression) : Generator[MethodBodyContext,
       Expression] = {
2
3      // turn model Expression into a CoGen expression
4      expr match {
5        case eq: EqualExpression => for {
6          left <- explore(eq.left)
7          right <- explore(eq.right)
8          intType <- toTargetLanguageType(TypeRep.Int)
9          e <- areEqual(intType, left, right)
10       } yield e
11
12       case ae: AdditionExpression => for {
13         left <- explore(ae.left)
14         right <- explore(ae.right)
15         e <- add(left, right)
16       } yield e
17
18       case se: SubproblemExpression => for {
19         self <- selfReference()
20         f <- getMember(self, names.mangle("helper"))
21         all_exprs <- forEach(se.args) { expr => for {
22             exp <- explore(expr)
23           } yield exp
24         }
25         res <- apply(f, all_exprs)
26       } yield res
27
28       case it:IteratorExpression => for {
29         args <- getArguments()
30       } yield args(it.iteratorNumber)._3
31
32       case lit:LiteralInt => for {
33         actual <- reify(TypeRep.Int, lit.literal)
34       } yield actual
35
36       case _ => ???     // detect when missing implementation
37     }
38   }
```

Listing 22: Explore acts as interpreter for model expressions

4.8.2 Model Workflow

When creating a model from an implemented solution, there are several ways it needs to be adjusted before it can properly fit into the model format. Firstly, any initialized values should be moved into the same place as the subproblem representation and differentiated using an if-else statement. For fibonacci this means taking the dp[0]=0 and dp[1]=1 lines into the loop. This allows any loop or recursion to use the full bounds from 0 up to the input value or array length. The next fix is to ensure that the base case is low and the final solution is high. For certain problems, especially knapsack problems, the array can be traversed in either direction, but for modeling purposes, it's necessary standardize this and always have the final answer be in the bottom right subproblem.

Once these adjustments have been made, the problem can then be converted to a model. To make the model, go through each field of the model constructor (problem, bounds, cases) and determine what value they should have, building up from smaller expressions when necessary.

The problem is simple, it's just the name of the problem you want to solve. The bounds are dependent on the dimensions of the problem and the input, for fibonacci the bounds looks like this:

```
List(new ArgExpression(0, "n", new IntegerType(), "i"))
```

bounds will always be a list with a number of elements equal to the dimensions of the problem. In the original model, the number of dimensions is always equal to the number of arguments to the problem for simplicity. In the case of fibonacci, there is one argument, the index for the argument is 0, the name of the argument is "n", the type of the argument is Integer, and the name of the iterator that uses this value as a bound is called "i". This tells the model parser that the problem will have one integer argument name "n". A bottom up implementation of this problem will have a loop that iterates from 0 to n, a top down implementation will use a recursive function with one argument that recurses on n. For a problem with more dimensions, the aditionally arguments would be expressed as additional ArgExpressions in the list.

The cases for a problem represent how to solve the problem given the problem coordinates. They are a series of if-else statements defined by a condition and an expression for the solution. In fibonacci the cases look like this:

```
List(
  (
    Some(new EqualExpression(i, zero)),
    zero
  ),
  (
    Some(new EqualExpression(i, one)),
    one
  ),
  (
    None,
```

```
    new SubproblemExpression(Seq(new SubtractionExpression(i, one))) +
    new SubproblemExpression(Seq(new SubtractionExpression(i, two)))
  )
)
```

The first case is the base case for when i is equal to zero, the second case is the base case for when i is equal to one, the final case is the else statement, and returns the recursive expression. For each case, the first element of the pair is the condition, the second element is the evaluated expression. In the final case, subproblem expressions are used to represent either recursive calls or references to the DP array. Note that in this example, i is defined as an iterator expression, and zero, one, and two are defined as literal integers with their respective values.

4.8.3 Model limitations

While the original model is very streamlined and can be used to generate many DP solutions, it is not sufficiently powerful enough to represent every DP problem. There are limitations in the original model that cannot be solved with a quick fix and require `EnhancedModel` to succesfully represent.

The biggest limitation with the model is that subproblem definitions cannot represent an arbitrary number of other subproblems. It's only possible to reference a constant number of previous subproblem solutions when finding a new solution. This is not sufficient to represent problems such as LONGESTINCREASINGSUBSEQUENCE where the solution at each index requires iterating over all previous subproblems.

Another limitation of the standard model is that the dimensions of the problem are tied to the input for that problem. There are several problems where the number of dimensions does not match the number of inputs. Knapsack problems often have extra inputs that shouldn't be turned into new dimensions for the solution. 0/1 KNAPSACK takes in two arrays for the weights and values of the items but only one of these inputs needs to be iterated over. Some problems have the opposite issue. With something like MAXSUBARRAY there's an extra dimension to be used for finding the max that doesn't correspond to any input for the problem.

The final limitation is that of ordering. Though it's rare, certain problems such as Matrix Chain Multiplication don't use the same simple ordering as most other problems and requires a nuanced definition to add this ordering to the model.

4.9 EnhancedModel

After successfully generating a few DP solutions, the initial modeling capabilities revealed they were inadequate to tackle more generic problems.

Consider the following two problems:

- MATRIXCHAINMULTIPLICATION asks to find the order in which to multiply a sequence of 2D matrices that minimizes the total costs. Given an array of integers A[] which represents the dimensions of matrices, compute the most efficient way to multiply these matrices together. The dimensions of the first matrix is stored in

A[0] and A[1]; the dimensions of the second matrix are stored in A[1] and A[2], and so on. In this case, the input is an array of integers, but that is simply the condensed representation of the matrices being multiplied.

- INTERLEAVINGSTRINGS is given three strings and asks to compute a boolean that represents whether `s3` is the interleaving of strings `s1` and `s2`.

In both cases, the goal is to find the problem decomposition, namely, framing the solution to a problem as the composition of solutions to smaller subproblems. In this case:

- For MATRIXCHAINMULTIPLICATION, $P(i, j)$ represents the minimum cost of multiplying the matrices using the dimensions from i to j. Note that $P(i, i) = 0$ as a base case, since a single dimension does not represent a matrix.

- For INTERLEAVINGSTRINGS, $P(i, j)$ is true if the first i characters of `s1` and the first j characters of `s2` can interleave to form the first `i + j` characters of `s3`. Here, $P(0, 0)$ since that represents empty strings as the base case. $P(0, i2)$ is true only if the character at position $i2$ in both $s2$ and $s3$ are the same. Similarly, $P(i1, 0)$ is true only if the character at position $i1$ in both $s1$ and $s3$ is the same.

`EnhancedModel` was necessary because the problem decomposition above $P(i, j)$ had nothing to do with the input to the problem.

```
class EnhancedModel(val problem:String,
    val input:Seq[ArgExpression],
    val subproblemType:LiteralExpression,   // prototype of the solution.
    val solutionType:LiteralExpression,     // prototype of return value
    val solution:SubproblemInvocation,
    val definition:Definition)
```

The input to the problem remains a sequence of `ArgExpression` but now the problem has its own separate definition that is independent of the structure of the problem.

The `solution` describes the subproblem that aligns with the original solution. For the INTERLEAVINGSTRINGS problem, the solution is defined as:

```
  val symTable = Map("i1" -> new StringLengthExpression(s1),
                     "i2" -> new StringLengthExpression(s2))
  val sol = new SubproblemInvocation(symTable, returnType = new BooleanType())
```

This captures the idea that the solution is found in the "lower right" of the solution space, namely $P(s1.length(), s2.length())$. Additionally, the return type of the solution is specified – in this case, the result is a Boolean value.

For MATRIXCHAINMULTIPLICATION, the solution is:

```
val symTable = Map("i" -> new LiteralInt(1),
                   "j" -> new ArrayLengthExpression(array) - one)
val sol = new SubproblemInvocation(symTable)
```

Which captures the notion that $P(1, nums.length - 1)$ represents the solution for computing $n - 1$ matrices when given n integer values. By using operator overloading, the above simplifies the expression that would have been used, namely: `new SubtractionExpression(new ArrayLengthExpression(array), one)`.

Below is the full modeling for Fibonacci:

```
val zero: LiteralInt = new LiteralInt(0)
val one: LiteralInt = new LiteralInt(1)
val two: LiteralInt = new LiteralInt(2)

val n = new ArgExpression(0, "n", new IntegerType(), "i")
val bound = List(n)

val i: HelperExpression = new HelperExpression("i")

val symTable = Map("i" -> n)
val sol = new SubproblemInvocation(symTable)

val oneCase = new IfThenElseDefinition(i == one,
  new ExpressionStatement(one),
  new ExpressionDefinition(
    new SubproblemExpression(Seq(i - one)) +
    new SubproblemExpression(Seq(i - two))))

val zeroCase = new IfThenElseDefinition(i == zero,
  new ExpressionStatement(zero),
  oneCase)

val Fib = new EnhancedModel("Fibonacci",
  bound,
  subproblemType = new LiteralInt(0),
  solutionType = new LiteralString(""),
  sol,
  zeroCase)
```

Compare this model with the earlier Fibonacci modeling. You can see that the problem definition is more structured – it is composed of a singular definition rather than a sequence of cases.

4.9.1 Enhanced Model Workflow

The workflow for creating an `EnhancedModel` for a given problem is very similar to the original model, with a noticeable increase in complexity. Like before, it is necessary to adapt the problem you're working with to fit into the model, though there is slightly more leniency in this regard because of the increased power of `EnhancedModel`. Because the order of subproblem traversal is manually defined in `EnhancedModel`, iteration can be done in either direction with the low or high value being the base case. It is still required to move default values into the loop.

Just like the original model, `EnhancedModels` can be built one argument at a time, though there are now additional arguments. The full list is `problem`, `input`, `subproblemType`, `solutionType`, `solution`, `definition`, `answer`, and `mode`.

`problem` is the same as in the original model, just a string for the name of the problem. Input is adapted from the bounds of the model, it represents the arguments that go into the problem. The input is identical to the bounds, a list of argExpression.

SubproblemType and SolutionType is where things actually become different. The additional power of `EnhancedModel` allows for more complicated problems. While the majority of problems always use helper methods that return ints and retrieves that return strings this is not always the case. In most cases these will remain as IntegerType() and StringType() respectively.

Solution is an argument that takes in a subproblem invocation. This represents the inputs for the helper method in the top-down case or the number of loops in the bottom up case and specifies how iteration variables should be mapped together. In the fibonacci `EnhancedModel`, the subproblem invocation looks like this:

```
SubproblemInvocation(Seq("i"), helpers = Map("i" -> i))
```

The `Seq("i")` means that the helper method for a top down will take only i as a parameter and there will be only one bottom up loop iterating over i. The Map creates a correspondence between the string "i" and the previously defined helper variable i.

The next argument is definition. This argument represents the formula for solving a subproblem. While the original model uses cases and automatically provides an if-else structure, the `EnhancedModel` allows for a more specific ordering of cases. Therefore, it allows users to specify which conditions are to be considered base cases. The caveat, however, is that if-else statements become nested both in model definition and the generated code. The definition for fibonacci looks like this:

```
IfThenElseDefinition(i == zero,
        ExpressionStatement(zero),
        IfThenElseDefinition(i == one,
            ExpressionStatement(one),
            ExpressionDefinition(new SubproblemExpression(Seq(i - one)) +
                new SubproblemExpression(Seq(i - two)))))
```

Fibonacci has 3 cases, so there are 2 nested `IfThenElseDefinitions`. Other than the explicit `IfThenElseDefinitions`, the biggest difference between the original model and `EnhancedModel` is the `ExpressionStatements` and `ExpressionDefinitions`. Because the

`EnhancedModel` is able to differentiate expressions from definitions, the expressions need to be wrapped so that they can be properly parsed.

Answer represents the final solution to the problem. Fibonacci's answer is this:
`ReturnExpressionDefinition(new SubproblemExpression(Seq(n))))`
In the top-down case, this is how the engine knows that the initial returned call to the helper method is `helper(n)`. In the bottom-up case, this becomes a final `return dp[n]` statement.

The final argument mode is used only for a small number of problems. For most bottom-up implementations it's best to start at the top left of the DP array and iterate rightward and downward, however there are certain problems where this is not the case. Matrix Chain Multiplication is one such problem where solutions are found along the central diagonal and in subsequent diagonals radiating outward. There needs to be some way of telling the engine which order subproblems should be solved. For a traditional problem, the canonical ordering suffices, for special cases, a tailor made ordering can be designed to fit the problem.

4.9.2 Comparison to Dynprog

With the addition of inner loops using `EnhancedModel`, the model is now capable of representing the same set of problems as Dynprog and more. While Dynprog uses higher order function manipulation in its subproblem expression, `EnhancedModel` allows a user to define complex loops that can achieve the same functionality in a more abstract way. Instead of finding the right sequence of nested higher order functions, it's possible to simply define a loop. `EnhancedModel` is even more powerful as it can have any number of dimensions within an inner loop which Dynprog would struggle with.

4.9.3 Limitations of Enhanced Model

Despite the added expressiveness of `EnhancedModel` there are still issues that limit its functionality. One big challenge with representing DP problems using `EnhancedModel` is that not all problems can be generated both top-down and bottom-up with the same `EnhancedModel`. Certain problems such as Matrix Chain Multiplication have flaws in their `EnhancedModel` representations that prevent them from having identical bottom-up and top-down representations. While most problems don't have this issue, it can invalidate the model's goal of abstracting the details of DP problems if it's necessary to manually code both versions.

Another issue with `EnhancedModel` is the added complexity. The additional fields and more nested expressions and statements increase the power of the model at the cost of simplicity. For many basic problems, it takes more time to represent them with the `EnhancedModel` than with the regular model because more thought needs to go into coming up with the right proper representation, especially with the if-else statements which need to be manually defined and nested with `EnhancedModel`.

A related issue to the complexity of `EnhancedModel` is redundancy. There are places in the `EnhancedModel` where it's required to define the same information more than once. For example there are multiple places in the `EnhancedModel` where you need to specify the names of the helper variables. Additionally, in a fully abstract representation of a

DP problem, none of the names of the arguments and iterators would matter. The only important detail is their types and ordering. While it might make it easier to understand which concepts are linked in the model, using variable names creates points in the model prone to user error if the names don't match.

5 Conclusion

At the conclusion of our project, we successfully identified and implemented 56 DP problems in our Java repository; of those, 21 are fully realized as `EnhancedModel` instances in CoGen.

Throughout the project, we authored several types of code. We maintained our family-oriented package structure in both our Java repository and the IL-DP branch of CoGen. In addition to our various implementations of DP problems, we compiled a suite of JUnit Jupiter [15] test cases, which serve as a reference when generating test cases in CoGen. These can be found in our Java repository [16]:

```
/Dynamic-Programming-MQP
└─ /app
   └─ /test
      └─ /org
         └─ /dp
```

The codebase developed during this project is also available for review [5]:

```
/expression-problem
└─ /helloWorld
   └─ /src
      └─ /main
         └─ /scala
            └─ /org
               └─ /combinators
                  └─ /archive
                     ├─ /unenhancedModels
                     └─ /cogen
                        ├─ /bottomUp
                        └─ /topDown
```

Within the `expression-problem` repository, `.../cogen` contains handwritten CoGen code used to generate solutions; DP problems implemented using the first iteration of our `Model` class can be viewed in `.../unenhancedModels`.

While organizing this extensive codebase, our understanding of the aforementioned "family-oriented" package structure evolved significantly, leading to key insights regarding how DP problems are fundamentally related.

5.1 Testing Code

We used multiple methods of testing code for correctness during this project. When working within the DP java repo we used `./gradlew test` to reach the handmade test files for each implemented problem. When working in CoGen we used `sbt test` from within the proper folder to use the generated `build.sbt` file to run generated test cases. These test cases were generated using the `..MainJava` files. Additionally, for testing all files at once you can use the glossary file to generate every implemented solution. These can similarly be tested using `sbt test`. When generating files it's essential to use either `fix_bottomup.py` or `fix_topdown.py` to add necessary imports for the files to run.

Model Tests:

Language	files	blank	comment	code
Scala	39	576	333	2493
Text	2	0	0	2
SUM:	41	576	333	2495

Model:

Language	files	blank	comment	code
Scala	41	473	314	1692
SUM:	41	473	314	1692

Java Repo:

Language	files	blank	comment	code
Java	56	797	1104	2556
SUM:	56	797	1104	2556

Figure 7: cloc results

5.2 Code Size Comparison

Using the `cloc` (count lines of code [9]) tool, we can see how many lines of code it took on average for both the original code and the model code. After using cloc in 3 locations to determine the size of both the java repo code and the CoGen model code and dividing by the number of implemented problems we can see that there are on average `2556/56=46` lines of code for standard implementations and `(2493+1692)/38=110` lines of code for model implementations. Model implementations are longer and take about twice as many lines of code, but when taking into account that they can generate both bottom-up and top-down and will be even more versatile in the future, this is a worthwhile trade.

5.3 Revisiting Families

Initially, we hoped that the similarities between DP problems of the same family would manifest in reusable code during the phases of model architecture and engine development. Instead, we noticed that they identified the structure of a problem rather than the semantic definition by subproblem construction. While developing the engine, we realized that the mapping of DP problems from `EnhancedModel` instances to CoGen and then again to Java relies only on exploring the problem's definitions recursively and maintaining information presented by instances of `Expression` and its subclasses. However, we decided to maintain the package structures in our repositories to give viewers a more digestible introduction to Dynamic Programming.

5.4 Future Work

While the current iteration of CoGen successfully demonstrates the viability of generating Java-based Dynamic Programming solutions from abstract `EnhancedModel` instances, the vast and varied landscape of DP algorithms leaves much room for expansion. Our initial implementation prioritized establishing a robust, foundational engine for standard problem types. Moving forward, the framework can be extended to support more

complex domain spaces, advanced recurrence patterns, and different data structures. The following sections outline key areas where our architecture could be enhanced to capture a more comprehensive subset of Dynamic Programming problems.

5.4.1 Research into Different Problem Structures

Although dynamic programming has very complex applications, we chose to limit our scope to problems with more structured inputs — such as integers, knapsacks, single sequences, two sequences, and grids–in order to better highlight the core patterns and recurrence structures that are common across the family of DP problems. Future work may include research into problems that require more complex inputs, such as graph-based DP problems where custom data structures are needed to represent nodes and edges, natural language processing with algorithms such as Viterbi, or image-based problems such as the Seam Carving algorithm, where the input is a 2D array of pixel values and the goal is to intelligently resize images by removing paths of least visual importance.

5.4.2 Additional Definition capabilities

The canonical structure of Dynamic Programming is to define the solution to a given subproblem in terms of solutions to smaller subproblems.

Some problems require more complicated conditional definitions. In the WORDBREAK problem, you are given a string, `s`, and `dict`, a dictionary of `n` words. The challenge is to detect whether `s` can be segmented into a concatenation of valid words from the dictionary. For example, the string `"catsandog"` cannot be formed by concatenating strings from the dictionary `["cats", "dog", "sand", "and", "cat"]`.

A top-down solution might be described as follows. The inner `for` loop iterates over every possible word in `dict` and returns `true` once found. If no such word is found, then `false` is returned by the helper.

```
public boolean compute() {
  return helper(0);
}

public boolean helper(int i) {
  if (i == s.length()) { return true; }

  for (int j = i + 1; j <= s.length(); j++) {
    String word = s.substring(i, j);
    for (int w = 0; w < dict.length; w++) {
      if (dict[w].equals(word) && helper(j)) {
        return true;    // Short Circuit
      }
    }
  }

  return false;
}
```

Listing 23: WORDBREAK top-down implementation

This subproblem structure offers the opportunity to "short circuit" once found; it also minimizes the number of times the helper method is invoked, since it is only called when `s.substrings(i, j)` is a word in the dictionary.

5.4.3 Additional Expression capabilities

While there are many expressions available to be used in the models, it would be impossible to make an exhaustive compendium of all expressions that can be used in DP problems. The choices we made for which expressions to include were purely dependent on which expressions were needed for the DP problems we implemented. For example, we didn't use bit-wise XOR as an expression because we hadn't found any problems that required it.

If one were to find a gap in expressivity that could be filled by an additional expression, they could implement a new expression by adding it as a new class that extends expression and writing a case in `explore()` that handles it.

This displays a potential issue with our models. Because they are several layers of abstraction away from java, any desired java expression needs to be manually implemented for it to function in the model. This is different from the DSL Dynprog where any R expression could be inserted into the representation of the problem.

5.4.4 Additional Array capabilities

Much like with the expression capabilities, our models don't have extensive array capabilities. However, unlike expression capabilities, the gaps are partially due to difficulty of implementation. One issue is that of arbitrary dimensioned arrays. `EnhancedModel` only has definitions for a small number of array dimensions. If a problem wanted to use a 5 or 6 dimensional array, those would need to be manually added to the engine. There's a similar limitation in CoGen where literal arbitrary dimensional arrays are difficult to generate.

Another shortcoming with array capabilities is that of iterating over subproblems. While `EnhancedModel` does have some hard-defined ways of accumulating subproblem solutions, they are specific and limited in their uses. If a user wanted a different kind of iteration, they would need to implement it themselves.

5.4.5 Solution Generation in Other Languages

Although the abstract nature of CoGen makes it capable of generating code in multiple languages, we limited our current implementation to Java. This decision was made to maintain parity with our hand-coded reference solutions, which were written exclusively in Java. Future research could expand upon this work by testing the engine's capability to generate solutions for DP problems in other target languages, such as Python or C++.

Testing our engine in other languages introduces the challenge of accounting for the target language's primary paradigm–specifically, whether it is functional or object-oriented. This distinction is critical, as it dictates how the generated code must handle state management, memoization, and overall program structure. Because our target output is Java, our current pipeline heavily relies on CoGen's object-oriented functionality. We recommend that future researchers explore the engine's functional capabilities by using the hand-coded CoGen implementations preserved in our archive as a baseline to generate solutions for functional languages. Utilizing other languages, such as Haskell, may aid in research on different search orders.

5.4.6 Path Traversal of Dynamic Programming Problems

With research into utilizing different languages comes the potential to explore subproblem traversal. While top-down approaches to DP solve sub-problems only as needed, bottom-up implementations follow specific search orders. We defined the "canonical" search order for bottom-up DP problems to be an iterative sweep through the DP matrix, implemented through the use of nested `for` loops. This ensures that all necessary subproblems are already solved before-hand, as most of the DP problems we encountered considered values at DP[I - 1][J], DP[I][J - 1], and DP[I - 1][J - 1]. However, problems such as MATRIXCHAINMULTIPLICATION cannot utilize this search order for its bottom-up approach; we discovered this when attempting to create a single `EnhancedModel` that allows for both top-down and bottom-up generation.

5.4.7 Researching Solution Path Retrievals

We had initially anticipated during the earlier phases of this project that giving our engine the ability to generate functions that reconstruct solution paths to DP problems would be trivial. However, as our understanding of Dynamic Programming deepened, we realized that while the conceptual goal is straightforward, generalizing its implementation is highly complex.

As discussed in Section 4, reconstructing a solution path reveals the sequence of decisions made to achieve the optimal value. This path is typically determined by backtracking from the final, largest subproblem. By re-evaluating the recurrence relation at each step, the algorithm identifies which preceding subproblem contributed to the current optimal result, iteratively adding these state transitions to the path list.

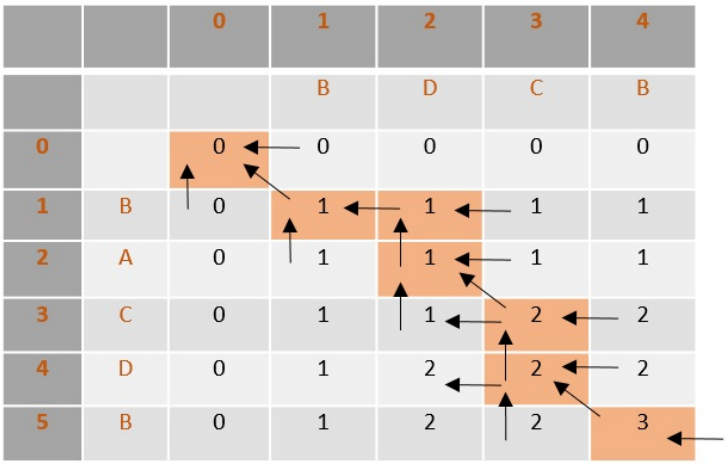

Figure 8: A traceback matrix for the LONGESTCOMMONSUBSEQUENCE algorithm. Image sourced from [33].

This can be easily visualized in most bottom-up implementations, as seen in Figure 8,

but paths can also be reconstructed from a top-down, memoized solution by tracing the cached dictionary entries.

Many forms of DP path retrievals we encountered returned values only pertinent to the problems they represented. For example, Listing 24, which is the retrieval for LONGESTCOMMONSUBSEQUENCE, returns the actual `String` representing the shared sequence of characters.

```
    // consistent storage for all decisions
    HashMap<Integer,Integer> decision = new HashMap<>();

    // Retrieve solution based on recursive traversal of decision points
    public String retrieve() {
       return expand(s1.length(), s2.length());
    }

    public String expand(int m, int n) {
        int len;
        if (dp != null) {
            len = dp[m][n];
        } else {
            len = computed_solution;
        }
        char[] path = new char[len];

        int r = m, c = n;
        int index = len - 1;

        while (r > 0 && c > 0) {
            int choice = decision.get(key(r, c));
            if (choice == 2) {          // from diagonal
                path[index] = s1.charAt(r - 1);
                index--;
                r--;
                c--;
            } else if (choice == 1) {   // from top
                r--;
            } else {
                c--;                    // from left
            }
        }

        return new String(path);
```

Listing 24: Path retrieval for LONGESTCOMMONSUBSEQUENCE using a cached decision map.

Instead, to generalize retrieval functions, we thought to make the return type a `String` that lists which subproblems to "take" to arrive at the final solution. Although the delivery of the solution path may change depending on the specific implementation, we highly recommend continuing with the idea of having retrieval functions return the sequence of substates. By abstracting the path into a list of subproblems, the engine can utilize a standardized generation template without needing to account for domain-specific return types, such as the constructed `String` in LONGESTCOMMONSUBSEQUENCE.

As demonstrated in Listing 25, the retrieval function reconstructs the solution path by starting at the global optimum and recursively navigating backward through the state space. By evaluating the transitions that occurred during the construction of the `dp` table, the algorithm conditionally concatenates strings such as "Take element x" to generate a step-by-step text representation of the chosen subproblems.

```java
  public String retrieve() {
    return findPath(nums.length-1, 1);
  }
  public String findPath(int x, int y) {
    if (y==0) {
      if (x==0) {
         return "Take element 0";
      } else {
        if (nums[x] > dp[x-1][0] + nums[x-1]) {
          return "Take element " + x;
        } else {
           return findPath(x-1, 0) + "\nTake element " + x;
        }
      }
    } else {
      if (x==0) {
        return findPath(0,1);
      } else {
        if (dp[x][0] > dp[x-1][1]) {
          return findPath(x ,0);
        } else {
          return findPath(x-1, 1);
        }
      }
    }
  }
}
```

Listing 25: Path retrieval for MAXSUBARRAY returning a `String` displaying the decision path.

A promising refinement to this strategy would be to transition the return type of `retrieve()` from a `String` to a structured collection, such as a `LinkedList` or a custom `Path` class. Returning a structured data type would allow the sequence of subproblems to be programmatically parsed and manipulated, rather than existing solely as human-readable text. However, implementing this method requires deeper research into COGEN's multi-language generation capabilities. The engine would need to either guarantee support for standard library collections across all target languages, or dynamically generate custom `Path` constructs using the appropriate object-oriented or functional paradigms for the chosen language.

References

[1] *2 Keys Keyboard*. LeetCode. Problem #650. URL: https://leetcode.com/problems/2-keys-keyboard/.

[2] Duncan Adamson et al. "Longest Common Subsequence with Gap Constraints". In: *arXiv preprint arXiv:2304.05270* (2023). URL: https://arxiv.org/abs/2304.05270.

[3] Richard Ernest Bellman. *The Theory of Dynamic Programming*. Tech. rep. P-550. RAND Corporation, 1954. URL: https://www.ams.org/journals/bull/1954-60-06/S0002-9904-1954-09848-8/S0002-9904-1954-09848-8.pdf.

[4] Vincent Boyer, Didier El Baz, and M. Elkihel. "Solution of Multidimensional Knapsack Problems via Cooperation of Dynamic Programming and Branch and Bound". In: *European Journal of Industrial Engineering* 4.4 (2010), pp. 434–449. DOI: 10.1504/EJIE.2010.035653. URL: https://homepages.laas.fr/elbaz/EJIE.pdf.

[5] Combinators et al. *Expression Problem*. https://github.com/combinators/expression-problem/tree/IL-DP. 2026.

[6] *Count Square Submatrices with All Ones*. LeetCode. Problem #1277. URL: https://leetcode.com/problems/count-square-submatrices-with-all-ones/.

[7] crh23. *Cantor's Pairing Function*. [Online; accessed Jan 28, 2026]. 2020. URL: https://commons.wikimedia.org/w/index.php?curid=91110360.

[8] Maxime Crochemore and Ely Porat. "Fast computation of a longest increasing subsequence and application". In: *Information and Computation* 208.9 (2010), pp. 1054–1059. ISSN: 0890-5401. DOI: 10.1016/j.ic.2010.04.003.

[9] Albert Danial. *cloc: Count lines of code in many programming languages*. Version 1.90. 2021. URL: https://github.com/AlDanial/cloc.

[10] *Decode Ways*. LeetCode. Problem #91. URL: https://leetcode.com/problems/decode-ways/.

[11] D. Descu. "Maximum Subarray Sum Problem". In: (). URL: https://personal.utdallas.edu/~daescu/maxsa.pdf.

[12] Erich Gamma et al. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley Longman Publishing Co., Inc., 1995.

[13] JavaParser Team. *JavaParser: A library for interacting with Java source code*. https://github.com/javaparser/javaparser. [Online; accessed 5-March-2026]. 2026.

[14] *Jump Game II*. LeetCode. Problem #45. URL: https://leetcode.com/problems/jump-game-ii/.

[15] JUnit Team. *JUnit 5 User Guide*. https://junit.org/junit5/. Accessed: 2026-03-03. 2026.

[16] Michael Lin, Matthew Samuels, and Charles Anton Sibal. *Dynamic Programming MQP*. https://github.com/cacsibal/Dynamic-Programming-MQP. 2026.

[17] David Maier. "The Complexity of Some Problems on Subsequences and Supersequences". In: *Journal of the ACM* 25.2 (1978), pp. 322–336. DOI: 10.1145/322063.322075. URL: https://link.springer.com/article/10.1007/BF01075212.

[18] Thomas Mailund. *Domain-Specific Languages in R: Advanced Statistical Programming*. Apress, 2018. ISBN: 1484235878. URL: https://amzn.to/2DRmFXb.

[19] *Maximum Size Sub-Matrix with All 1s in a Binary Matrix*. GeeksforGeeks. URL: https://www.geeksforgeeks.org/dsa/maximum-size-sub-matrix-with-all-1s-in-a-binary-matrix/.

[20] *Min Cost Climbing Stairs*. LeetCode. Problem #746. URL: https://leetcode.com/problems/min-cost-climbing-stairs/description/.

[21] *Minimum ASCII Delete Sum for Two Strings*. LeetCode. Problem #712. URL: https://leetcode.com/problems/minimum-ascii-delete-sum-for-two-strings/description/.

[22] *Minimum Cost For Tickets*. LeetCode. Problem #983. URL: https://leetcode.com/problems/minimum-cost-for-tickets/.

[23] *Minimum Path Sum*. LeetCode. Problem #64. URL: https://leetcode.com/problems/minimum-path-sum/.

[24] *Minimum Time to Make Rope Colorful*. LeetCode. Problem #1578. URL: https://leetcode.com/problems/minimum-time-to-make-rope-colorful/description/.

[25] Shin-Cheng Mu and Tung-Ju Chiang. "Longest segment of balanced parentheses: An exercise in program inversion in a segment problem". In: *Journal of Functional Programming* 31 (2021), e31. DOI: 10.1017/S0956796821000253.

[26] *N-th Tribonacci Number*. LeetCode. Problem #1137. URL: https://leetcode.com/problems/n-th-tribonacci-number/description/.

[27] Saul B. Needleman and Christian D. Wunsch. "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins". In: *Journal of Molecular Biology* 48.3 (1970), pp. 443–453. DOI: 10.1016/0022-2836(70)90057-4.

[28] *Ones and Zeroes*. LeetCode. Problem #474. URL: https://leetcode.com/problems/ones-and-zeroes/.

[29] *Perfect Squares*. LeetCode. Problem #279. URL: https://leetcode.com/problems/perfect-squares/description/.

[30] *Scalameta*. https://scalameta.org/. Accessed: 2024-01-29.

[31] *Solving Questions With Brainpower*. LeetCode. Problem #2140. URL: https://leetcode.com/problems/solving-questions-with-brainpower/.

[32] *Triangle*. LeetCode. Problem #120. URL: https://leetcode.com/problems/triangle/description/.

[33] Tutorials Point. *Longest Common Subsequence Algorithm*. https://www.tutorialspoint.com/data_structures_algorithms/longest_common_subsequence.htm. Accessed: 2026-03-04.

[34] *Uncrossed Lines*. LeetCode. Problem #1035. URL: https://leetcode.com/problems/uncrossed-lines/.

[35] *Unique Paths*. LeetCode. Problem #62. URL: https://leetcode.com/problems/unique-paths/.

[36] Robert A. Wagner and Michael J. Fischer. "The String-to-String Correction Problem". In: *Journal of the ACM* 21.1 (1974), pp. 168–173. DOI: 10.1145/321796.321811.

[37] Wikipedia contributors. *Fibonacci sequence — Wikipedia, The Free Encyclopedia*. [Online; accessed 5-March-2026]. 2026. URL: https://en.wikipedia.org/wiki/Fibonacci_sequence.